WEB SYSTEMS DESIGN – PROJECT 6

VISTULA

UNIVERSITY

Name: Amjad Massaoud

Student ID: 78709

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

1. Introduction 3

2. Project Assumptions and Objectives 3

3. Technological Foundations 3

4. Implementation Process 4

4.1 Project Implementation Screenshots 4

1. Route List Verification 4

2. Migration File for short_links Table 4

3. ShortLink Model 5

4. Base62 Helper 6

5. ShortLinkController 7

6. RedirectController 8

7. Updated api.php Routes 8

8. web.php Redirect Route 9

9. Successful POST Request 9

10. Successful GET List Request 9

11. Validation Error - Invalid URL 10

12. Validation Error - Missing Album Number 10

13. Redirect Response - 302 Found 10

14. Missing Short Code - 404 Not Found 10

15. Redis DBSIZE After Cache Usage 11

16. Redis PING 11

17. Session 6 Documentation File 11

18. README Update 12

19. Git Commit 12

4.2 Conceptual Explanations 13

4.3 Troubleshooting Technical Issues 13

5. Conclusion 14

6. References 14

1. Introduction

The purpose of this laboratory project is to extend the existing web system with a new backend module: a URL shortener. In the previous laboratory stages, the project was already developed as a modern web system using a Laravel backend, Vue frontend, PostgreSQL database, Redis cache, Docker Compose deployment, Nginx reverse proxy, API versioning, and standardized JSON responses.

Project 6 focuses on adding a practical feature that is commonly used in real web applications. A URL shortener allows users to submit a long URL and receive a shorter URL that can be shared more easily. When the short URL is opened, the system redirects the user to the original long URL.

Although this functionality looks simple from the user perspective, it requires several important technical decisions. The system must validate the original URL, generate a unique short code, store the mapping in the database, perform fast redirects, count the number of redirects, and handle invalid codes safely. This makes the URL shortener module a useful example of web systems design because it combines API design, persistence, validation, redirection, caching, and error handling.

2. Project Assumptions and Objectives

The main objective of this project was to add a URL shortener module to the existing Laravel API without breaking the previous task management API. The new module had to work under the already existing versioned API prefix that includes the student number: `/api/78709/v1`.

The main objectives of this laboratory were:

- Verify that the existing Docker-based system still works.
- Create a new database table named short_links.
- Create a ShortLink model for database interaction.
- Create a Base62 helper to generate short readable codes.
- Create an API controller for listing, creating, and showing short links.
- Create a redirect controller for /r/{code}.
- Register the new API routes in routes/api.php and the redirect route in routes/web.php.
- Validate user input, especially the original URL and album number.
- Store URL mappings in PostgreSQL and cache the list endpoint using Redis.
- Test successful POST and GET requests, validation errors, redirects, missing codes, and Git commit evidence.

3. Technological Foundations

PHP: The backend programming language used to implement server-side logic.

Laravel: The backend framework used for routing, controllers, validation, Eloquent ORM, database migrations, JSON responses, and redirect handling.

PostgreSQL: The relational database system used to store the original URLs, short codes, click counts, and album number.

Redis: The in-memory data store used for caching the short link list endpoint.

Docker Compose: The tool used to run the complete system in containers, including backend, frontend, PostgreSQL, and Redis.

Nginx: The web server and reverse proxy used to expose the system and forward API and redirect requests.

Base62 Encoding: An encoding technique that converts numeric database IDs into short readable strings.

Git: The version control system used to track and commit the implementation changes.

cURL: The command-line tool used to test endpoints and verify HTTP status codes.

4. Implementation Process

4.1 Project Implementation Screenshots

The following screenshots document the implementation and verification process for Project 6. Each screenshot shows either an implementation file or a test result proving that the URL shortener module works correctly.

1. Route List Verification

```
jad@fedora-2:~/uni-project/project6/backend$ php artisan route:list
GET|HEAD   api/78709/v1/short-links ..................... short-links.index › Api\ShortLinkController@index
POST       api/78709/v1/short-links ..................... short-links.store › Api\ShortLinkController@store
GET|HEAD   api/78709/v1/short-links/{short_link} ........ short-links.show › Api\ShortLinkController@show
GET|HEAD   api/78709/v1/tasks ........................... tasks.index › Api\TaskController@index
POST       api/78709/v1/tasks ........................... tasks.store › Api\TaskController@store
GET|HEAD   r/{code} ..................................... RedirectController
Showing [19] routes
```

The route list shows that the short link API endpoints and the /r/{code} redirect route are registered correctly.

2. Migration File for short_links Table

2026_08_30_220000_create_short_links_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('short_links', function (Blueprint $table) {
            $table->id();
            $table->string('original_url', 2048);
            $table->string('short_code')->unique()->nullable();
            $table->unsignedBigInteger('click_count')->default(0);
            $table->string('album_number');
            $table->timestamps();
        });
    }
};
```

The migration file creates the short_links table with original_url, short_code, click_count, album_number, and timestamps.

3. ShortLink Model

ShortLink.php — backend > app > Models > ShortLink.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class ShortLink extends Model
{
    protected $fillable = [
        'original_url',
        'short_code',
        'click_count',
        'album_number',
    ];
}
```

The ShortLink model defines fillable fields for safe mass assignment through Eloquent.

4. Base62 Helper

Base62.php — backend > app > Support > Base62.php

```php
<?php

namespace App\Support;

class Base62
{
    private const ALPHABET = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ';

    public static function encode(int $number): string
    {
        if ($number === 0) {
            return self::ALPHABET[0];
        }

        $base = strlen(self::ALPHABET);
        $result = '';

        while ($number > 0) {
            $remainder = $number % $base;
            $result = self::ALPHABET[$remainder] . $result;
            $number = intdiv($number, $base);
        }

        return $result;
    }
}
```

The Base62 helper converts numeric database IDs into short readable codes using digits, lowercase letters, and uppercase letters.

5. ShortLinkController

ShortLinkController.php — app/Http/Controllers/Api/ShortLinkController.php

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\ShortLink;
use App\Support\Base62;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;

class ShortLinkController extends Controller
{
    public function index(): JsonResponse
    {
        $links = Cache::remember('short-links.index', 60, function () {
            return ShortLink::query()->latest()->get();
        });

        return response()->json(['data' => $links], 200);
    }

    public function store(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'original_url' => ['required', 'url', 'max:2048'],
            'album_number' => ['required', 'string', 'max:50'],
        ]);

        $shortlink = ShortLink::create([
            'original_url' => $validated['original_url'],
            'album_number' => $validated['album_number'],
        ]);

        $shortlink->short_code = Base62::encode($shortlink->id);
        $shortlink->save();

        Cache::forget('short-links.index');

        return response()->json([
            'data' => $shortlink,
            'short_url' => url('/r/' . $shortlink->short_code),
        ], 201);
    }
}


```

The ShortLinkController handles listing, creating, and showing short links through the versioned API.

6. RedirectController

RedirectController.php — app/Http/Controllers/RedirectController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\ShortLink;
use Illuminate\Http\Request;

class RedirectController extends Controller
{
    public function __invoke(Request $request, string $code)
    {
        $shortLink = ShortLink::query()->where('short_code', $code)->first();

        if (!$shortLink) {
            return response('Short link not found.', 404);
        }

        $shortLink->increment('click_count');

        return redirect()->away($shortLink->original_url, 302);
    }
}
```

The RedirectController finds a short code, increments click_count, and redirects the browser to the original URL.

7. Updated api.php Routes

api.php — backend > routes > api.php

```php
<?php

use App\Http\Controllers\Api\ShortLinkController;
use App\Http\Controllers\Api\TaskController;
use Illuminate\Support\Facades\Route;

Route::prefix('78709/v1')->group(function () {
    Route::apiResource('tasks', TaskController::class);
    Route::apiResource('short-links', ShortLinkController::class)
        ->only(['index', 'store', 'show']);
});
```

The api.php file registers the short-links resource under the versioned prefix /api/78709/v1.

8. web.php Redirect Route

web.php — backend > routes > web.php

```php
<?php

use App\Http\Controllers\RedirectController;
use Illuminate\Support\Facades\Route;

Route::get('/r/{code}', RedirectController::class);
```

The web.php file registers the /r/{code} redirect route because browser redirects are not JSON API endpoints.

9. Successful POST Request

```
jad@fedora-2:~/uni-project/project6$ curl -i -X POST http://127.0.0.1:8080/api/78709/v1/short-links -H "Content-Type: application/json"
-d '{"original_url":"https://laravel.com/docs/routing","album_number":"78709"}'
HTTP/1.1 201 Created
Content-Type: application/json

{"data":{"original_url":"https:\/\/laravel.com\/docs\/routing","album_number":"78709","updated_at":"2026-08-
30T16:25:46.000000Z","created_at":"2026-08-30T16:25:46.000000Z","id":1,"short_code":"1"},"short_url":"http:\/\/127.0.0.1:8080\/r\/1"}
```

The POST request creates a short link and returns HTTP 201 Created with a generated short URL.

10. Successful GET List Request

```
jad@fedora-2:~/uni-project/project6$ curl http://127.0.0.1:8080/api/78709/v1/short-links
{"data":
[{"id":1,"original_url":"https:\/\/laravel.com\/docs\/routing","short_code":"1","click_count":0,"album_number":"78709","created_at":"2026-
08-30T16:25:46.000000Z","updated_at":"2026-08-30T16:25:46.000000Z"}]}
```

The GET request lists created short links and returns HTTP 200 OK.

11. Validation Error - Invalid URL

```
jad@fedora-2:~/uni-project/project6$ curl -i -X POST http://127.0.0.1:8080/api/78709/v1/short-links -H "Content-Type: application/json"
-H "Accept: application/json" -d '{"original_url":"not-a-valid-url","album_number":"78709"}'
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json

{"message":"The original_url field must be a valid URL.","errors":{"original_url":["The original_url field must be a valid URL."]}}
```

The API rejects an invalid URL and returns HTTP 422 with a validation error for original_url.

12. Validation Error - Missing Album Number

```
jad@fedora-2:~/uni-project/project6$ curl -i -X POST http://127.0.0.1:8080/api/78709/v1/short-links -H "Content-Type: application/json"
-H "Accept: application/json" -d '{"original_url":"https://example.com"}'
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json

{"message":"The album_number field is required.","errors":{"album_number":["The album_number field is required."]}}
```

The API returns HTTP 422 when album_number is missing from the request.

13. Redirect Response - 302 Found

```
jad@fedora-2:~/uni-project/project6$ curl -i http://127.0.0.1:8080/r/1
HTTP/1.1 302 Found
Location: https://laravel.com/docs/routing
Content-Type: text/html; charset=UTF-8
```

The /r/1 endpoint performs a real redirect and returns HTTP 302 Found with a Location header.

14. Missing Short Code - 404 Not Found

```
jad@fedora-2:~/uni-project/project6$ curl -i http://127.0.0.1:8080/r/doesnotexist
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=UTF-8

Short link not found.
```

The /r/doesnotexist endpoint returns HTTP 404 Not Found when the short code does not exist.

15. Redis DBSIZE After Cache Usage

```
jad@fedora-2:~/uni-project/project6$ docker compose exec redis redis-cli -n 1 DBSIZE
(integer) 0
jad@fedora-2:~/uni-project/project6$ curl http://127.0.0.1:8080/api/78709/v1/short-links
{"data":[{"id":1,"original_url":"https:\/\/laravel.com\/docs\/routing","short_code":"1","click_count":1,"album_number":"78709"}]}
jad@fedora-2:~/uni-project/project6$ docker compose exec redis redis-cli -n 1 DBSIZE
(integer) 1
```

Redis DBSIZE was checked after calling the short-links list endpoint to verify cache-related behavior.

16. Redis PING

```
jad@fedora-2:~/uni-project/project6$ docker compose exec redis redis-cli PING
PONG
```

The Redis PING command returned PONG, confirming that Redis was reachable.

17. Session 6 Documentation File

session6-url-shortener.md — docs > session6-url-shortener.md

```
# Session 6 - URL Shortener Module

## Purpose of the module
This module allows users to create short URLs from long URLs and redirect users from a short code to the original URL.

## Functional requirements
- The system accepts a long URL and returns a short URL.
- The system redirects users from a short code to the original URL.
- The system validates the original URL.
- The system stores URL mappings in PostgreSQL.
- The system counts redirects using click_count.

## Endpoints
POST /api/78709/v1/short-links
GET /api/78709/v1/short-links
GET /api/78709/v1/short-links/{id}
GET /r/{code}
```

The documentation file explains the module purpose, requirements, Base62 encoding, endpoints, and testing evidence.

18. README Update

README.md

```
55  ## URL Shortener Module
56
57  The project now includes a simple URL shortener module.
58
59  Current endpoints:
60  - POST /api/78709/v1/short-links
61  - GET /api/78709/v1/short-links
62  - GET /api/78709/v1/short-links/{id}
63  - GET /r/{code}
64
65  The module uses PostgreSQL for persistent storage, Redis for caching the list endpoint, and Base62 encoding for generating short cod
66
```

The README was updated with the URL Shortener Module section and its endpoints.

19. Git Commit

```
jad@fedora-2:~/uni-project/project6$ git commit -m "Project 6: add URL shortener module"
[master 25dd26d] Project 6: add URL shortener module
10 files changed, 209 insertions(+), 3 deletions(-)
create mode 100644 backend/app/Http/Controllers/Api/ShortLinkController.php
create mode 100644 backend/app/Http/Controllers/RedirectController.php
create mode 100644 backend/app/Models/ShortLink.php
create mode 100644 backend/app/Support/Base62.php
create mode 100644 backend/database/migrations/2026_08_30_220000_create_short_links_table.php
create mode 100644 docs/session6-url-shortener.md
```

The final Project 6 changes were committed to Git with a descriptive commit message.

4.2 Conceptual Explanations

What a URL Shortener Does:
A URL shortener receives a long URL and creates a shorter URL that points to the same destination. When a user opens the short URL, the system finds the original URL in the database and redirects the browser to it. In this project, the /r/{code} route performs the redirect.

Why URL Validation Is Necessary:
URL validation is necessary because the system should only store valid web addresses. If invalid values are accepted, redirects may fail or users may be sent to broken destinations. Laravel validation rejects invalid values with HTTP 422.

Why Short Codes Must Be Unique:
Short codes must be unique because each code must point to exactly one original URL. If duplicate codes existed, the system would not know which destination to use. The database enforces uniqueness on the short_code column.

What Base62 Encoding Is:
Base62 encoding converts numeric values into short strings using 62 characters: digits, lowercase letters, and uppercase letters. In this project, the database ID is encoded into the short_code.

Why Redirect Uses HTTP 302:
HTTP 302 Found tells the browser that the requested resource is temporarily located at another URL. This is suitable for the shortener because the redirect is dynamic and controlled by the application.

Why Redis Cache Is Useful:
Redis improves performance by storing frequently requested data in memory. The short link list endpoint uses Cache::remember, and the cache is cleared after creating a new short link.

Functional Requirements:
The system accepts a long URL, validates it, stores the mapping, generates a unique short code, returns a short URL, lists links, shows one link, redirects to the original URL, counts redirects, and handles invalid input.

Non-Functional Requirements:
The redirect should be fast, short codes should be unique, responses should be consistent, status codes should be correct, Redis should support caching, and documentation should make the module understandable.

4.3 Troubleshooting Technical Issues

1. Redirect Route Through Nginx:
During testing, the /r/{code} route had to be handled by the Laravel backend instead of the frontend application. The Nginx configuration was adjusted so that /r/ requests are proxied to the backend. After rebuilding the web container, the redirect endpoint correctly returned HTTP 302 Found with the original URL in the Location header.

2. Missing Short Code Response:
When testing a missing short code, the controller was adjusted to safely return a 404 Not Found response. This ensures that invalid short codes do not result in application crashes and are handled predictably.

3. Redis Cache Verification:
Redis was tested using PING and DBSIZE. The PING result confirmed that Redis was reachable. The list endpoint was also called before checking DBSIZE to verify cache-related behavior.

4. Docker Rebuild After Code Changes:

After changing controllers, routes, and the Nginx configuration, Docker containers had to be rebuilt using `docker compose up -d --build`. This ensured that the updated implementation was active inside the containers.

5. Conclusion

In this project, I successfully extended the existing Laravel web system by adding a URL shortener module. The module allows users to create short links from long URLs, list existing short links, view a selected short link, and redirect users from a short code to the original URL.

The implementation included a new database migration, an Eloquent model, a Base62 helper, an API controller, and a redirect controller. The new endpoints were added under the existing versioned API prefix **/api/78709/v1**, while the browser redirect route was added separately under **/r/{code}**.

The system validates user input, stores URL mappings in PostgreSQL, generates unique short codes, increments click counts during redirects, uses Redis cache for the list endpoint, and handles errors such as invalid URLs and missing short codes. Overall, this laboratory improved the system by adding a practical web feature and connecting important web systems design concepts including persistence, validation, uniqueness, caching, HTTP redirects, and functional and nonfunctional requirements.

6. References

1. Laravel Documentation - Routing: https://laravel.com/docs/routing
2. Laravel Documentation - Controllers: https://laravel.com/docs/controllers
3. Laravel Documentation - Validation: https://laravel.com/docs/validation
4. Laravel Documentation - Eloquent ORM: https://laravel.com/docs/eloquent
5. Laravel Documentation - Migrations: https://laravel.com/docs/migrations
6. Laravel Documentation - Cache: https://laravel.com/docs/cache
7. Laravel Documentation - Responses and Redirects: https://laravel.com/docs/responses
8. MDN Web Docs - HTTP Status Codes: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
9. Docker Documentation - Docker Compose: https://docs.docker.com/compose/
10. Redis Documentation: https://redis.io/docs/latest/

Project 6 – Git Repository & Version Control Verification

All project modules, backend controllers, models, database migrations, automated test suites, frontend components, and compiled assignment reports have been committed and synchronized with the remote GitHub repository **Jadtales/uni-projects** on branch `master`.

```
jad@fedora-2:~/uni-project$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean

jad@fedora-2:~/uni-project$ git log -1 --oneline
96dbd38 (HEAD -> master, origin/master) - projects
```